How to Den

`Den.pm` is a Perl module to convert denaturant titration melt data from "timed" titrations or individual wavelength scans at each titration point (see later example). Below is a sample test script, `test_den.pl`, to process a titration that contains two different segments of titration points, which were collected in two data files on a fluorimeter, `072808A.PRN` and `072808B.PRN`:

```
use Den;
my $d = Den -> new()
            -> open_parameter_file('den_parameter_file.txt')
            -> conjoin_files('072808A.PRN', '072808B.PRN', 'conjoined.txt')
            -> calc_den()
            -> extract_signal()
            -> save_data('072808_MELT.txt');
```

A key file is the parameter file
This file is opened using the method, `open_parameter_file($some_filename)`. Here is an example, `den_parameter_file.txt`, used in the case above:

```
CUVETTE_VOLUME          2000
TITRANT_CONC            7.02
CUVETTE_CONC            0
TITRATION_PROGRAM       30_25_48_64
TIME_INIT               106
TIME_PER_BIN            106
TIME_START              0
TIME_END                8374
TIME_OFFSET             3180
```

This parameter file is in a KEY VALUE format. These keys are all UPPER CASE format. The file generally uses tab or spaces to delimit, but the user can specify a different delimiter as the second argument if desired (see below). These keys refer to individual scalar properties in the Den module, where the parameters are stored. All keys may be accessed by the all lower case method gets and sets, i.e.:

```
my $tp = $d -> titration_program(); #get the titration program string
$d -> titration_program($tp); #set the titration program string
```

This would get the variable, `$tp`, in the former case and set the property to `$tp` in the latter. Sure you could just set all those parameters in the `test_den.pl` script this way if you'd like to do it that way and not open the parameter file at all.

The above script, `test_den.pl`, is fancy where two segments were collected. `TITRATION_PROGRAM`, states these two segments are as follows in its shorthand form, i.e., `30_25_48_64`:

```
SEG       PTS.   VOLUME (UL)
============================
SEG1      30     25
SEG2      48     64
============================
```

NOTE the `TITRATION_PROGRAM` must separate the values for the segments using only an underscore and no spaces or other characters—only numbers and underscores.

CUVETTE_VOLUME, TITRANT_CONC, and CUVETTE_CONC should be self-explanatory.

Extracting the signal with the `extract_signal` method

TIME_INIT is the time prior to the first titration operation (usually where the cuvette is in 0 M denaturant or something like that. TIME_INIT is generally the same as the TIME_PER_BIN, which is the time taken between titration steps.

The parameters TIME_START and TIME_END are basically unnecessary, although originally they allowed the algorithm to exclude data before time start and after time end during the signal extraction process.

The extract_signal method only requires these parameters (TIME_PER_BIN and TIME_INIT) and some raw data to work off of to extract the signal. So maybe you'd like to open up a file of raw data first, using open_raw_data($some_file):

```
use Den;
my $d = Den -> new()
            -> open_parameter_file('den_parameter_file.txt')
            -> open_raw_data('072808A.PRN')
            -> extract_signal();
```

The method can take the parameters for init and bin times in the method as arguments, but this is not needed if you set them in the parameter file or separately above:

```
$d -> extract_signal($time_init, $time_per_bin);
my @signal_array = $d -> signal();
```

The signal is then stored into the array property, signal, accessed simply as above.

Conjoining files for two different segments

```
conjoin_files($file_1_to_join, $file_2_to_join, $conjoined_filename,
$time_offset, $delimiter);
my @raw = $d -> raw_data();
```

Here TIME_OFFSET is important to time shift the second file when stitching them together with the conjoin_files method. The offset may be set in the parameter file or as an argument here. On yeah and the delimiter can be set with $d -> delimiter(chr(9)) if you like tab delimited format, although tab is default. The raw data is accessed using the raw_data property.

Calculating the denaturant concentration during the titration

If the TITRATION_PROGRAM was set along with CUVETTE_VOLUME, TITRANT_CONC, and CUVETTE_CONC, then the denaturant concentrations can be calculated using the calc_den method:

```
$d -> calc_den($titrant_conc, $cuvette_conc, $cuvette_volume,
$titration_program);
my @denaturant_array = $d -> den();
```

Remember that those args are not needed if the parameters were set in the parameter file. So simply calc_den() will do. To get the array of denaturant concentration data, use the den property.

Accessing the data

You can get the complete dataset of the denaturant concentrations and respective spectroscopic signals from the module using the data property:

```
$d -> calc_den();
$d -> extract_signal();
my @data = $d -> data();
```

Example for a simple, single-segment melt

For this case in the script, `test_den1.pl`, the data is contained in a single file, `01.PRN`.

```
use Den;
my $d = Den -> new()
            -> open_parameter_file('den_parameter_file.txt')
            -> open_raw_data('01.PRN')
            -> calc_den()
            -> extract_signal()
            -> save_data('01_MELT.txt');
```

This is the parameter file, `den_parameter_file_1.txt`, for the simple case above.

```
CUVETTE_VOLUME          2000
TITRANT_CONC            7.02
CUVETTE_CONC            0
TIME_INIT               29.5
TIME_PER_BIN            29.5
TITRATION_PROGRAM       153_25
VERBOSE                 1
DEN_NAME                GDM
SIGNAL_NAME             FL
HEADER                  1
```

Example of denaturant titration where each individual titration point is a wavelength scan

For this case, the data are contained in a list of text files containing the wavelength scans of the titration. To keep it easy the files are named in numerical order with a simple file extension. For example, a list of files could be `0.txt`, `1.txt`, `2.txt`, `3.txt`, `4.txt` ... `60.txt` for a 60 point titration which includes the first point (i.e., `0.txt`) before the titration has started. The files can be delimited using tabs or commas or spaces. Just set that parameter in the parameter file. If you do not set the delimiter then tab is used by default.

There are new property parameters for processing a list of wavelength scans. Here is `den_parameter_file_3.txt`, for example, which will process four wavelength scans for a FRET experiment. Donor and acceptor wavelengths are given where the signals will be extracted over a wavelength width of 5 nm. In the example, the wavelength files (`0.txt`, `1.txt`, `2.txt` and `3.txt`) all have the extension, `txt`, which is set in the property, `FILE_SPECTRUM_EXTENSION`. If the spectrum files have headers, then set the `FILE_SPECTRUM_HEADER_LENGTH` to the number of lines to exclude from the beginning of the file prior to accessing the spectrum wavelength and signal data. The

header in the example spectra is three lines long. N.B. the FLUORESCENCE_MODE is set to FRET for this experiment so the signals from the two emission wavelengths are extracted and ratioed to give a FRET value for each titration point during processing.

```
CUVETTE_VOLUME                2000
TITRANT_CONC                  7.02
CUVETTE_CONC                  0
TITRATION_PROGRAM             3_100
FLUORESCENCE_MODE             FRET
WAVELENGTH_DONOR              510
WAVELENGTH_ACCEPTOR           520
WAVELENGTH_WIDTH              5
FILE_SPECTRUM_EXTENSION       txt
FILE_SPECTRUM_HEADER_LENGTH   3
DEN_NAME                      GDM
SIGNAL_NAME                   FRET
HEADER                        1
VERBOSE                       1
```

Here is the Perl script which will use the Den.pm module to process the wavelength scans for a FRET experiment.

```
use Den;
my $den = Den -> new()
              -> open_parameter_file('den_parameter_file_3.txt')
              -> calc_den()
              -> extract_wavelength_signal()
              -> save_data('spectrum_FRET_MELT.txt');
```

Notice a new method was called to extract the wavelength signal, i.e., extract_wavelength_signal(). This is the major processing change from a timed titration.

In the next example, a titration made up of wavelength scans that is processed for a single fluorescence wavelength can be done by changing the FLUORESCENCE_MODE property in the parameter file to the value, SINGLE_WAVELENGTH. Here is den_parameter_file_2.txt, for example:

```
CUVETTE_VOLUME                2000
TITRANT_CONC                  7.02
CUVETTE_CONC                  0
TITRATION_PROGRAM             3_100
FLUORESCENCE_MODE             SINGLE_WAVELENGTH
WAVELENGTH                    510
WAVELENGTH_WIDTH              5
FILE_SPECTRUM_EXTENSION       txt
FILE_SPECTRUM_HEADER_LENGTH   3
VERBOSE                       1
DEN_NAME                      GDM
SIGNAL_NAME                   FL
HEADER                        1
```

In the above example the WAVELENGTH property is set where the emission signal will be extracted over the 5 nm WAVELENGTH_WIDTH.

Below is the Perl script which will use the Den.pm module to process the wavelength scans for a single wavelength emission.

```
use Den;
my $den = Den -> new()
              -> open_parameter_file('den_parameter_file_2.txt')
              -> calc_den()
              -> extract_wavelength_signal()
              -> save_data('spectrum_MELT.txt');
```

In the next example, the parameter file is delimited by commas, but the spectra files are delimited by tabs. This issue can be circumvented by setting the delimited argument in `open_parameter_file` method and `extract_wavelength_signal` method. Here is the parameter file, `den_parameter_file_4.txt`:

```
CUVETTE_VOLUME,               2000
TITRANT_CONC,                 7.02
CUVETTE_CONC,                 0
TITRATION_PROGRAM,            3_100
FLUORESCENCE_MODE,            FRET
WAVELENGTH_DONOR,             510
WAVELENGTH_ACCEPTOR,          520
WAVELENGTH_WIDTH,             5
FILE_SPECTRUM_EXTENSION,      txt
FILE_SPECTRUM_HEADER_LENGTH,  3
DEN_NAME,                     GDM
SIGNAL_NAME,                  FRET
HEADER,                       1
VERBOSE,                      1
```

Here is the corresponding script, `test_den4.pl`.

```
use Den;
my $den = Den -> new()
              -> open_parameter_file('den_parameter_file_4.txt', chr(44))
              -> calc_den()
              -> extract_wavelength_signal('FRET', 'txt', chr(9))
              -> save_data('spectrum_FRET_2_MELT.txt');
```

The code `chr(44)` given for the delimiter argument is the ASCII value for comma; `chr(9)` is the delimiter ASCII argument for tab. The `'FRET'` argument is the `FLUORESCENCE_MODE`. The `'txt'` argument is for the file extensions of all the spectra files. Since the last delimiter argument was tab, then in `save_data` will save the output file with tab set as the delimiter. If you wanted to set the delimiter back to comma, then simply pass `chr(44)` as the second argument in `save_data`:

```
use Den;
my $den = Den -> new()
              -> open_parameter_file('den_parameter_file_4.txt', chr(44))
              -> calc_den()
              -> extract_wavelength_signal('FRET', 'txt', chr(9))
              -> save_data('spectrum_FRET_2_MELT.txt', chr(44));
```

Example of denaturant titration where all the titration's wavelength scans are stored in one large csv text file

For this case using the method, `extract_batch_wavelength_signal`, the source data are contained in a single ASCII csv text file (comma-separated values). In the csv file, the odd-numbered columns are the wavelength values, and the even-numbered columns are the corresponding intensity signal values. Here is an truncated excerpt from the example cvs data source file, `Exp1-2-9-2023.csv`:

```
0,,1,,2,
Wavelength (nm),Intensity (a.u.),Wavelength (nm),Intensity (a.u.),Wavelength
(nm),Intensity (a.u.)
500,4.782354832,500,4.663383007,500,4.480920792
501.0599976,5.35133934,501.0599976,5.259993076,501.0599976,5.024883747
501.9599915,5.751307964,501.9599915,5.789194107,501.9599915,5.554028034
503.0299988,6.425584316,503.0299988,6.55336237,503.0299988,6.082288265
503.9299927,6.885650158,503.9299927,7.088843822,503.9299927,6.83702898
505,7.416787624,505,7.743680477,505,7.455962658
506.0599976,8.136991501,506.0599976,8.378785133,506.0599976,8.125063896
506.9599915,8.778132439,506.9599915,9.01288414,506.9599915,8.676625252
508.0299988,9.415640831,508.0299988,9.811053276,508.0299988,9.414240837
508.9299927,9.820152283,508.9299927,10.26308537,508.9299927,10.01785946
510,10.5374403,510,11.10376072,510,10.66672421
```

To extract the signal, the method, `extract_batch_wavelength_signal`, takes four arguments: `$file, $header, $fluorescence_mode, $delimiter`, which are the filename of the csv file, the number of lines in the header, the fluorescence mode (either 'FRET' or 'SINGLE_WAVELENGTH), and the delimiter data record separator (a comma by default for a csv file). Here is an example script, which uses the method:

```
use Den;
my $den = Den -> new()
              -> open_parameter_file('2-9-2023-den_parameter_file.txt')
              -> calc_den()
              -> extract_batch_wavelength_signal('Exp1-2-9-2023.csv', 2,
'FRET', chr(44))
              -> save_data('2-9-2023-MELT.txt');
```

The parameter file (`2-9-2023-den_parameter_file.txt`) additionally sets the donor and acceptor wavelengths and the wavelength width, which is defines a wider averaging window than a single wavelength. This is the parameter file:

```
CUVETTE_VOLUME              2500
TITRANT_CONC                7.108
CUVETTE_CONC                0
TITRATION_PROGRAM           40_25_40_70
FLUORESCENCE_MODE           FRET
WAVELENGTH_DONOR            520
WAVELENGTH_ACCEPTOR         570
WAVELENGTH_WIDTH             5
DEN_NAME                    GDM
SIGNAL_NAME                 FRET
HEADER                      1
VERBOSE                     1
```

An excerpt of the final melt output in the file, `2-9-2023-MELT.txt`, is:

```
GDM              FRET
0                2.31241083549878
0.07108          2.1622260249952
0.1414492        1.95818237206906
0.211114708      1.78492419745733
0.28008356092    1.59735520351474
0.3483627253108  1.48748542735946
0.41595909805769 1.36751653693792
0.48287950707711 1.25191629054288
0.54913071200634 1.18499674413249
0.61471940488628 1.11716129573301
0.67965221083741 1.05090864649804
0.74393568872904 0.988411476931763
0.80757633184175 0.93769728670379
0.87058056852333 0.889280823926893
0.93295476283810 0.837759392816793
0.99470521520972 0.804692208527889
1.05583816305762 0.768562326674956
1.11635978142705 0.733116682382585
…
```

Another csv file format option is when one column (the first column) contains the wavelengths and all columns thereafter contain the corresponding signal intensities for each titration point. The method that extracts these signals is called `extract_z_batch_wavelength_signal`. The letter z denotes that these signal intensities are basically a z-axis for the raw data. The method takes the same four arguments as `extract_batch_wavelength_signal`, i.e., `$file`, `$header`, `$fluorescence_mode`, and `$delimiter`. Here is an example script:

```
use Den;
my $den = Den -> new()
              -> open_parameter_file('2-10-2023-den_parameter_file.txt')
              -> calc_den()
              -> extract_z_batch_wavelength_signal('Exp1-2-10-2023.csv', 3,
'FRET', chr(44))
              -> save_data('2-10-2023-MELT.txt');
```

Performing a ligand titration experiment instead of a denaturant titration

For this case, we use the method, `calc_ligand`, which takes the following arguments: `$ligand_stock_concentrations`, `$ligand_titration`, `$cuvette_volume`, `$cuvette_ligand_concentration`, and `$ligand_name`. The ligand stock concentrations is an array reference to an array or list of the stock ligand concentrations used in the titration (see example below). The ligand titration is a reference to an array of arrays of the titration volumes on the points at each stock concentration (see example below). The cuvette volume in the volume of the cuvette generally in microliters (you should use the same units in the ligand titration. The cuvette volume can also be set in the parameter file (see example below). The cuvette ligand concentration is the starting ligand concentration (use the same units as the ligand stock concentrations—see example below). The ligand name is the name of ligand used; it acts as a header of the ligand concentrations in the saved data file. Here is an example script followed by the parameter file.

First, the script:

```
use Den;
my $ligand_stock_concentrations = [ 15.38, 153.8, 1538.0, 15380 ];
#concentrations in micromolar
my $ligand_titration = [ [1, 2.5, 6], [1, 2.5, 6], [1, 2.5, 6], [1, 2.5, 6] ];
#volumes in microliter
my $den = Den -> new()
              -> open_parameter_file('3-1-2023-den_parameter_file_new.txt')
              -> calc_ligand($ligand_stock_concentrations, $ligand_titration)
              -> extract_batch_wavelength_signal('Exp1-3-1-2023.csv', 2,
'SINGLE_WAVELENGTH', chr(44))
              -> save_data('3-1-2023-output.txt');
```

The stock concentration list is pretty self-explanatory. The ligand titration is a reference to an array of arrays. Each array in the array of arrays is a list of volumes titrated for each stock concentration. In this case 1, 2.5 and 6 microliters are delivered for each stock concentration for a total of twelve titration points plus the starting point (generally zero micromolar), yielding 13 total points.

Here is the corresponding parameter file:

```
CUVETTE_VOLUME                      2500
CUVETTE_LIGAND_CONCENTRATION        0
FLUORESCENCE_MODE                   SINGLE_WAVELENGTH
WAVELENGTH                          570
WAVELENGTH_DONOR                    520
WAVELENGTH_ACCEPTOR                 570
WAVELENGTH_WIDTH                    5
LIGAND_NAME                         PGA_20MER
SIGNAL_NAME                         FL570
HEADER                              1
VERBOSE                             1
```

Notice like the `$ligand_titration`, a parameter which must be set in the script file (since it is a reference to an array of arrays), the cuvette volume (`CUVETTE_VOLUME`) is also in microliters (here set as 2500). The starting ligand concentration (`CUVETTE_LIGAND_CONCENTRATION`) in the cuvette is set to 0 in units of micromolar just as the ligand stock concentrations above. Note again that the stock concentrations are a reference to an array and cannot be set in the parameter file but must be instead passed as arguments to `calc_ligand` in the script file. The starting cuvette concentration does not need to be zero, but often it is. The `LIGAND_NAME` can also be set in the parameter file.

As the titration progresses, the cuvette volume will grow by the total volume titrated. Also the cuvette ligand concentration will change reflecting the increase in concentration of ligand throughout the titration.

How to Hoh

Now you can be a data farmer. The Hoh module (`Hoh.pm`) can manipulate datasets to make an output file, table, or hash of hashes (a hoh). Use as a typical Perl module:

```
use Hoh;
$hoh =  Hoh->new();
```

Data can be input as text files that are delimited by tabs, commas, or whatever. Be sure to set the property of the delimiter if not using tabs, i.e., tab is default:

```
$hoh -> delimiter(',');
```

File format for your data should have the first row as the list of data columns. Each column name must be unique single word entries or data will be overwritten by the last instance of a repeated name. These column names are keys to the hoh (see below). Don't use spaces in the names. All Caps is a good convention at this point. Then each row starts with a unique name. Consider the file, `my_data.txt`:

```
MY_KEYS   CNAME1 CNAME2 CNAME3 ...
UNIQUE_1  Bob    Harry  Moe
UNIQUE_2  George John   12345
UNIQUE_AB Pi     Beta   Kappa
...
```

Also if the data in your table has spaces in the individual entries, you should either use quotes around the entry or use underscores. For example, the entry, Mr. Rogers, should be either Mr._Rogers or "Mr. Rogers" to be handled properly.

A file may be opened to make a hoh using the `file_to_hoh` method:

```
my %opened_hoh = $hoh -> file_to_hoh('my_data.txt');
```

`open_hoh` is basically like `file_to_hoh`, except `open_hoh` returns the module and can be used like this:

```
my $hoh_obj = $hoh -> new() -> open_hoh('my_data.txt');
```

You could set the filename and delimiter those properties prior to opening if you like.

```
$hoh -> file('my_data.txt');
$hoh -> delimiter(',');
$hoh -> open_hoh();
```

Now you may access data in the hoh in this way, which retrieves 'George' in the example below in `$datum`:

```
my ($row, $col) = ('UNIQUE_2', 'CNAME1');
my $datum = $opened_hoh{ $row }{ $col };
```

If you want to resave the hoh after some modification, then you must place it back in the `hoh` property and use the `save` method.

```
$opened_hoh{ $row }{ $col } = $something_else;
$hoh -> hoh(%opened_hoh);
$hoh -> save('my_new_data.txt');
```

You can save certain specified column slices of the data in a specific order by using the `print_order` property:

```
$hoh -> print_order(qw(CNAME2 CNAME1));
```

In the above example, `CNAME2` will be printed before `CNAME1`. A print order array can also be loaded from a file, where the file contains a whitespace separated list is single word column names to print in the exact order listed.

```
$hoh -> open_print_order_file($my_filename);
```

Without `print_order` defined, the whole hoh is printed in alphabetical order.

To join two files together where row keys may overlap can be accomplished either by joining two hohs together of similar data structure or opening files. The two hashes must be passed as references to hashes, of course:

```
my $href_a = \%hash_a;
my $href_b = \%hash_b;
$hoh -> join_hohs($href_a, $href_b);
```

The joined Hoh can be accessed using:

```
my %hoh = $hoh -> hoh();
```

I think that Santa says the same thing around Christmas. Also files can be opened and joined and saved using a single method:

```
$hoh -> join_hohs($hoh_file_a, $hoh_file_b, $joined_hoh_file_out,
$print_order_file);
```

Don't worry if you leave out an output file name. The default output file is `outfile.txt`. Also remember that the print order may be set in the property `print_order` as a list or opened separately. See example below. You can even sort the rows alphabetically by a specified column name using the `sort_by` property:

```
$hoh -> sort_by('CNAME2');
```

If you don't specify the `sort_by` property the data are output by alphabetical order using the row key column.

Example script that joins two files together, and then joins to a third file. The print order is defined by a `qw()` list or any array:

```
use Hoh;
my $h = Hoh->new();
$h -> sort_by('AANUM');

my @list = qw(AANUM DDGdD70 DDGdD40ERR DDGDD40 DDGDD70ERR);

$h -> print_order(@list);
$h -> join_files('ddgdd40.txt', 'ddgdd70.txt');

#Note default output file from join_files is outfile.txt

@list = qw (AANUM DDGNI DDGNIERR DDGDD40 DDGDD40ERR DDGDD70 DDGDD70ERR);

$h -> print_order(@list);
$h -> join_files('outfile.txt', 'FRET_MELT_DDGS.txt', 'FRET.txt');
```

Hoh.pm can also make Chimera attribute files to display data onto a protein structure. See http://www.cgl.ucsf.edu/chimera/ for more details. Make a simple text file where the first column contains a unique list of residue numbers (i.e. 1, 2, 100, 101) or a list of residue types (i.e. GLY, LYS, PHE) followed by columns of data to used to generate the attribute. Simply use the `save_as_chimera_attribute` method. Note the first argument must be a single word with not underscores or any punctuation, where the first letter is always lowercase (a weird Chimera issue); for example, 'phiValue' is good but not 'PhiValue' or 'Phi-value'! If you don't, it will create an appropriate name in the right format for you. The second argument contains the column name key to be exported; for these attributes only one column may be specified as the attribute:

```
use Hoh;
my $hoh_obj = Hoh -> new() -> open_hoh($hoh_file, $delimiter) ->
save_as_chimera_attribute($chimera_attribute_name, $col_with_attribute);
```

For example:

```
my $hoh_obj = Hoh->new()->open_hoh('PHICHIMERA.txt',',') ->
save_as_chimera_attribute('phiValue2','PHI');
```

The Chimera attribute will be saved as a `yourChimeraAttributeName.txt` (whatever your name was) with the .txt extension so you may look at it easily in notepad or some simple text editor.

Here is an example of an output Chimera attribute file, called `phiValue.txt`:

```
# From Krantz Lab Hoh.pm
# Chimera attribute output file for residues
# Text file is PHICHIMERA.txt
attribute: phiValue
match mode: 1-to-1
recipient: residues
    :39    0.5707
    :40    -0.56007
    :62    0.23958
    :70    0.93064
    :112   0.37229
    :119   1.59486
    :126   0.31066
    :129   2.52437
    :145   0.32538
    :147   0.49904
    :148   0.26554
    :154   3.28114
    :155   0.09889
    :174   0.01373
    :213   0.5012
    :215   0.13245
    :217   0.00406
    :220   0.14255
    :221   0.34891
    :245   0.01833
    :247   0.37886
```

…Happy hohing…